

Prompt para Claude Code — Trade Performance Dashboard

Pega este documento completo como primer mensaje en Claude Code. Está organizado por fases: construye, revisa y avanza. No intentes el one-shot.

0. Rol y contexto

Actúa como un **ingeniero full-stack senior especializado en aplicaciones de datos para negocio**, con criterio de producto. Vamos a construir **Trade Performance Dashboard**: una app web sencilla, profesional y *local-first* para que profesionales de **trade marketing, category management y ventas** midan el desempeño comercial por **SKU, categoría, canal, campaña y tienda** a partir de archivos simples de ventas (Excel/CSV).

El usuario final **no es técnico**. Es un analista comercial, un trade marketer o un estudiante de diplomado. La interfaz va en **español**, con terminología de práctica (sell-out, rotación, participación, ticket, margen, góndola, canal). El tono visual: limpio, ejecutivo, confiable. Nada de jerga de developer en la UI.

Principio rector: los datos del usuario **nunca salen del navegador**. Sin backend, sin login, sin base de datos en servidor. Privacidad como argumento de venta.

1. Stack y restricciones técnicas

- **React 18 + Vite + TypeScript.**
- **100% client-side.** Todo el procesamiento (parseo, cálculo, hallazgos, export) ocurre en el navegador.
- **Tailwind CSS** para estilos. Diseño sobrio y profesional (no plantilla genérica): tipografía clara, jerarquía visual fuerte, tarjetas de KPI, buen uso del espacio en blanco.
- **Parseo de archivos:** **xlsx** (SheetJS) para Excel y **papaparse** para CSV.
- **Gráficos:** **recharts**.
- **Persistencia local:** los proyectos y su estado se guardan en **IndexedDB** (usa **idb** o **dexie**). El usuario puede cerrar y volver a sus proyectos. (Nota: en este entorno usa IndexedDB; si lo despliegue en un sandbox sin storage, degrada a estado en memoria con aviso.)

- **Export PDF:** usa `react-to-print` (imprimir a PDF) para el resumen ejecutivo — es lo más confiable. Export de tablas a Excel/CSV con SheetJS.
- **Deploy objetivo: Vercel.** La app se publica en Vercel como sitio estático conectado al repositorio de GitHub. El despliegue es automático en cada push a `main`. Incluye instrucciones de build (`vite build`, output `dist/`) y configuración de Vercel en el README. Asegúrate de que el proyecto buildee limpio para Vercel (sin dependencias de Node en runtime, todo client-side).
- **Repositorio de GitHub:**
<https://github.com/lagotre/Herramienta-Trade-Performance-Dashboard.git>

No construyas: autenticación, multi-tenant, servidor, API propia, ni integraciones externas. Mantén el alcance disciplinado.

2. Modelo de datos (contrato canónico)

La app trabaja internamente con este esquema. El archivo del usuario **no tiene que coincidir** — habrá una pantalla de mapeo de columnas (Fase 1).

Campo interno	Tipo	Requerido	Notas
<code>fecha</code>	Date	Sí	Acepta múltiples formatos; normaliza a ISO.
<code>canal</code>	string	Sí	Normalizar (ver validación).
<code>tienda</code>	string	No	Punto de venta.
<code>categoria</code>	string	Sí	
<code>subcategoria</code>	string	No	
<code>marca</code>	string	No	
<code>sku</code>	string	Sí	
<code>producto</code>	string	No	Nombre del producto.
<code>unidades</code>	number	Sí	

Campo interno	Tipo	Requerido	Notas
venta	number	Sí	Venta en dinero.
margen	number	No	Margen/utilidad en dinero.
precio_promedio	number	No	Si no viene, se calcula = venta/unidades.
campana	string	No	Campaña/promoción asociada.
inventario	number	No	Inventario disponible.
num_transacciones	number	No	Tickets/transacciones . Habilita ticket promedio.

Degradación elegante (crítico): si falta **margen**, oculta KPIs y hallazgos de rentabilidad con un tooltip que explique por qué. Si falta **inventario**, sustituye **rotación** por **velocidad de venta** (unidades/día). Si falta **num_transacciones**, no muestres "ticket promedio"; muestra solo "precio promedio". Nunca rompas el dashboard por una columna ausente.

3. Fórmulas de KPIs (explícitas)

- **Ventas (\$):** Σ **venta**.
- **Unidades:** Σ **unidades**.
- **Margen (\$):** Σ **margen** (si existe).
- **Margen %:** Margen \$ ÷ Ventas \$.
- **Precio promedio:** Ventas \$ ÷ Unidades.
- **Ticket promedio:** Ventas \$ ÷ **num_transacciones** (solo si la columna existe).
- **Participación (share):** Venta del segmento ÷ Venta total, por dimensión (SKU, categoría, canal, etc.).
- **Crecimiento:** comparación periodo vs. periodo. **Lógica:** si el archivo contiene ≥ 2 periodos distintos (agrupa **fecha** por mes), calcula crecimiento mes contra mes. Si solo hay un periodo, muestra el KPI como "requiere histórico" y permite **cargar un segundo archivo como periodo base** para comparar. No inventes crecimiento con un solo periodo.

- **Rotación:** Unidades vendidas ÷ Inventario promedio (si hay inventario). Si no, **Velocidad de venta** = Unidades ÷ días del periodo.
-

4. Motor de hallazgos automáticos (el diferenciador)

Implementa un **motor de reglas con umbrales configurables** (un objeto **config** editable, porque el criterio del trade marketer debe poder ajustarlos). Cada hallazgo se presenta como una **frase accionable y priorizada** (severidad: alta / media / baja), con el dato exacto que la respalda.

Reglas mínimas:

1. **Concentración (Pareto):** si el top 20% de SKUs concentra $\geq 80\%$ de ventas $\rightarrow$ "Alta concentración: X SKUs generan el Y% de las ventas. Riesgo de dependencia." Reporta el % real.
2. **SKUs ganadores:** top decil por venta \$ y margen % por encima de la mediana $\rightarrow$ destácalos.
3. **SKUs lentos / cola muerta:** cuartil inferior por unidades, o ventas cero en el periodo, o rotación bajo umbral (si hay inventario).
4. **Categoría en caída:** crecimiento $< -X\%$ (default -10%) mes contra mes (requiere ≥ 2 periodos).
5. **Sobredependencia de canal:** si un canal concentra $> 60\%$ (configurable) de las ventas $\rightarrow$ riesgo de concentración de canal.
6. **Trampa de margen:** segmento con alto volumen (cuartil superior de ventas) pero margen % bajo la mediana $\rightarrow$ "Buen volumen, baja rentabilidad."
7. **Campaña sin impacto:** mide *uplift* de las filas con **campana** vs. baseline comparable (mismas categorías/canales sin campaña). **Sé honesto en el copy:** es una señal direccional de incrementalidad, no conversión pura (no tenemos datos de tráfico).
8. **Inventario en riesgo:** alto inventario + baja rotación (si hay inventario).

Cada hallazgo termina en una **acción sugerida** en lenguaje de negocio (ej.: "Diseñar flash sale en WhatsApp Business para liquidar SKUs de cola y liberar caja").

5. Validación y salud de datos (pre-dashboard)

Antes del dashboard, muestra un **"Tablero de salud de datos"** con severidad (bloqueante / advertencia) y permite continuar:

- SKUs sin categoría (conteo + muestra).

- Productos/filas duplicadas (mismo **sku** + **fecha** + **canal** + **tienda**).
 - **Canales mal escritos**: normaliza contra una lista canónica (Tienda Física, E-commerce, WhatsApp Business, Marketplace, Redes Sociales, Email/CRM) y **sugiere fusiones** (ej.: "Whatsapp", "whats app", "WA" → WhatsApp Business). El usuario aprueba o edita.
 - Fechas no parseables o inconsistentes.
 - Ventas negativas o vacías.
 - Categorías sin margen.
 - Campañas sin identificación.
-

6. Flujo de la app (pantallas)

Paso 1 — Crear proyecto: nombre (ej. "Análisis ventas mayo 2026 – Canal retail moda"), empresa/cliente, periodo, moneda, tipo de negocio, canales a analizar. Se guarda en IndexedDB.

Paso 2 — Carga de datos: subir Excel/CSV + botón "**Cargar datos de ejemplo**" (ver Fase de demo). Pantalla de **mapeo de columnas**: detecta automáticamente columnas por nombre, deja al usuario reasignar con dropdowns. Esta pantalla es la más delicada de la UX — dedícale cuidado.

Paso 3 — Salud de datos: el tablero de validación de la sección 5.

Paso 4 — Resumen ejecutivo: dashboard con KPIs generales (Ventas, Unidades, Margen, Crecimiento, Top categorías, Top canales, Top SKUs) en tarjetas + gráficos.

Paso 5 — Análisis específicos: pestañas **SKUs / Categorías / Canales / Campañas / Tiendas / Tendencias**. Filtros globales: periodo, canal, categoría, marca, campaña.

Paso 6 — Hallazgos detectados: la sección del motor de reglas (sección 4), priorizada.

Paso 7 — Exportar: resumen ejecutivo (PDF), resumen para cliente, lista de acciones, rankings (Excel).

Lógica central a respetar: **Datos** → **limpieza** → **análisis** → **hallazgos** → **decisiones** → **acciones**.

7. Dataset demo (para auto-testear)

Genera un dataset sintético realista de **Angel Moda Jeans** (marca colombiana de jeans para mujer, caso recurrente): ~12 SKUs de jeans/denim, 5 canales (2 Tiendas Físicas, E-commerce, WhatsApp Business, Marketplace), 3 meses (marzo–mayo 2026), algunas campañas asociadas. **Incluye a propósito unas filas sucias** (un canal mal escrito, un SKU sin categoría, una venta negativa) para que el tablero de salud de datos tenga algo que mostrar. Cablea este dataset al botón "Cargar datos de ejemplo" y úsalo para verificar que cada fase funciona end-to-end.

8. Plan de construcción por fases

Construye **una fase a la vez** y detén-te para que yo revise antes de seguir:

- **Fase 0:** Scaffold Vite + TS + Tailwind, layout base, navegación por pestañas, estados vacíos. **Al cerrar esta fase**, inicializa Git, agrega un `.gitignore` apropiado (node_modules, dist, .env) y haz **push al repositorio remoto** <https://github.com/lagotre/Herramienta-Trade-Performance-Dashboard.git> en la rama `main` (commit inicial: "Fase 0: scaffold base"). A partir de aquí dejo el repo conectado a Vercel para despliegue automático, así que cada fase siguiente termina también con su commit y push.
 - **Fase 1:** Ingesta (upload + parseo + mapeo de columnas + botón de datos demo).
 - **Fase 2:** Tablero de salud de datos (validaciones).
 - **Fase 3:** Motor de KPIs + dashboard de resumen ejecutivo.
 - **Fase 4:** Pestañas de análisis + filtros globales.
 - **Fase 5:** Motor de hallazgos.
 - **Fase 6:** Exportaciones.
 - **Fase 7:** Pulido: estados de error/vacío, copys en español, accesibilidad, responsive, README con build y deploy.
-

9. Criterios de aceptación

- Cargo el dataset demo y veo KPIs, las 6 pestañas y al menos 5 hallazgos accionables sin un solo error en consola.
- Subo un Excel con columnas distintas y el mapeo funciona; las columnas opcionales ausentes degradan con elegancia (sin pantallas rotas).
- El tablero de salud de datos detecta el canal mal escrito, el SKU sin categoría y la venta negativa del demo.

- Exporto el resumen ejecutivo a PDF y un ranking a Excel.
 - Cierro y reabro el navegador y mi proyecto sigue ahí.
 - Toda la UI está en español, con terminología de trade marketing.
-

10. Roadmap (no construir ahora, solo dejar el gancho)

Deja el código preparado para una v2 donde los **hallazgos** se envíen a la API de Anthropic y un modelo los narre como una **recomendación ejecutiva en voz de consultor de trade marketing**. No lo implementes en el MVP; solo no cierres puertas en la arquitectura.

Empecemos por la Fase 0. Cuando termines, haz el push inicial al repositorio de GitHub indicado, muéstrame qué construiste y espera mi visto bueno antes de la Fase 1.